

Báo cáo cuối kỳ: SCRFD, SPOS-NAS và ASR+JSAR

23127333 Trương Quốc Cường 23127011 Lê Anh Duy 23127199 Trần Gia Huy

CSC14006 – Nhận dạng
Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM

TP. Hồ Chí Minh, ngày 20 tháng 4 năm 2026



Mục lục

- 1 Mở đầu
- 2 SCRFD Gốc
- 3 Reproduction
- 4 SPOS-NAS Cho CR
- 5 ASR+JSAR Cho SR Và Train
- 6 Kết luận

Mở đầu

Phạm vi của báo cáo này

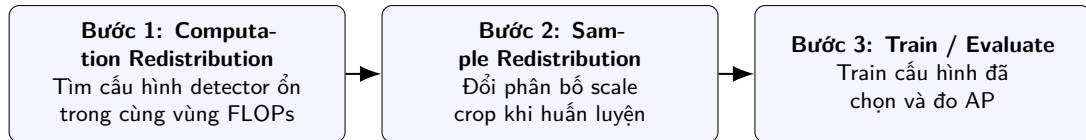
- Báo cáo chỉ tập trung vào bài báo **SCRFD** và pipeline detector một giai đoạn cho face detection trên **WIDER FACE**.
- Nội dung được tổ chức theo đúng logic của một report bài báo khoa học: trình bày phương pháp, tái hiện bằng official code, đối chiếu paper với code, rồi phân tích kết quả.
- Từ đó nhóm mở rộng theo đúng hai mắt xích quan trọng của SCRFD:

Hai hướng mở rộng

SPOS-NAS cho bước **Computation Redistribution**;

ASR+JSAR cho bước **Sample Redistribution** và assignment khi train.

SCRFD nên được nhìn như pipeline 3 bước



- Điểm quan trọng là ba bước này gắn chặt với nhau: kiến trúc đúng nhưng sample policy sai vẫn tụt Hard AP.
- Vì vậy phần mở rộng cũng phải đi theo đúng trình tự **CR trước, SR sau**.

SCRFD Gốc

- SCRFD hướng vào **face detection hiệu quả**, đặc biệt là nhóm **tiny faces** trong ảnh VGA.
- Trên WIDER FACE, split Hard chứa rất nhiều mặt cực nhỏ, nên detector không thể chỉ mạnh về ngữ nghĩa sâu; nó còn phải giữ tốt chi tiết không gian ở các tầng nông.
- Bài báo vì thế không chỉ hỏi mô hình nào mạnh hơn, mà hỏi hai câu đúng hơn:

Hai câu hỏi cốt lõi

Với cùng ngân sách FLOPs, compute nên đặt ở đâu?

Và trong lúc train, detector nên nhìn thấy khuôn mặt ở những scale nào nhiều hơn?

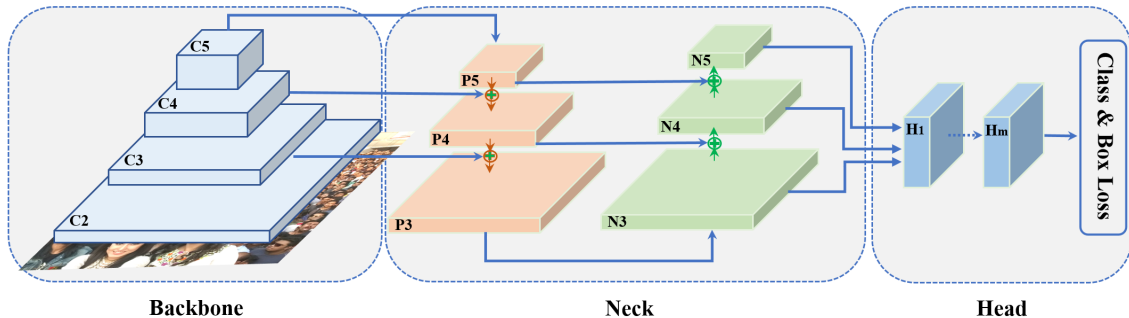
Minh họa WIDER FACE Hard và tiny faces



Kiến trúc nền Backbone–Neck–Head

- **Backbone:** tạo chuỗi đặc trưng $C_2 \rightarrow C_5$.
- **PAFPN:** hợp nhất thông tin nhiều mức để vừa giữ vị trí vừa giữ ngữ nghĩa.
- **Dense head:** dự đoán classification và bbox trên các feature map.
- Trọng tâm của SCRFD là làm cho cấu trúc này phù hợp hơn với tiny-face detection thay vì bê nguyên detector phân loại ảnh sang.

Minh họa kiến trúc Backbone–Neck–Head



Computation Redistribution được làm theo hai pha

- **Pha 1: search backbone.** Mục tiêu là cô lập câu hỏi riêng cho backbone: trong khung 4 stage C_2, C_3, C_4, C_5 , compute nên dồn nhiều hơn vào stage nào.
- Ở repo public demo, bước này mặc định sinh **64 candidate backbone**; paper mô tả quy mô đầy đủ là **320 candidate**.
- Mỗi candidate đều bị lọc lại theo cùng vùng chi phí, trong code là khoảng **target FLOPs $\pm 2\%$** , nên chỉ những cấu hình còn quanh 2.5G mới được giữ lại để train và so sánh.
- Các cấu hình hợp lệ sau đó được **train rút gọn 80 epoch**, đánh giá bằng **Hard mAP**, rồi chọn **config tốt nhất** làm template cho pha 2.
- **Không có bước lấy top 10% hay khoảng tin cậy thống kê**; đây là ranking-based search: sinh candidate $\rightarrow$ lọc FLOPs $\rightarrow$ train ngắn $\rightarrow$ chọn config thắng.

Các bậc tự do mà CR thực sự đi tìm

- **Pha 1 – search backbone:** không search từ số 0, mà đi trong một khung 4 stage cố định rồi thay đổi các bậc tự do chính của backbone.
- Các bậc tự do ở bước này là: **loại block, base width ban đầu, độ sâu từng stage, và tỉ lệ độ rộng giữa các stage.**
- Nói ngắn gọn, pha 1 đang đi tìm **hình dạng backbone:** stage nào sâu hơn, stage nào rộng hơn, và độ dốc tăng width giữa $C_2 \rightarrow C_5$ nên như thế nào.
- **Pha 2 – search toàn mạng:** từ backbone template thắng ở pha 1, paper không mở toàn bộ detector một cách tự do, mà chỉ cho backbone dao động quanh template đó rồi tinh chỉnh tiếp **neck và head.**
- Các bậc tự do ở bước này là: **mức scale lại độ sâu backbone, mức scale lại độ rộng backbone, số kênh của neck, số kênh của head, và độ sâu của head.**
- Ý nghĩa của cách tách này là tránh để neck/head che mờ tín hiệu thật sự của backbone ở đầu quy trình, rồi chỉ sau đó mới phân bổ nốt compute cho toàn detector.

Quy trình tìm kiếm kiến trúc CR



Giả định và giới hạn của CR

- CR không search từ số 0. Paper giả định trước một **khung detector cố định**: backbone 4 stage, PAFPN, dense head, các mức stride chính vẫn giữ nguyên.
- Trong pha backbone, search thực chất là phân bổ lại depth/width quanh template ban đầu, chứ không mở tự do mọi loại module.
- Trong pha toàn mạng, detector cũng không được mở hoàn toàn; backbone đã bị khóa quanh template thẳng, rồi mới phân phối nốt compute cho neck và head.
- Vì vậy giá trị của CR không phải ở chỗ paper search mọi thứ, mà ở chỗ paper chọn đúng abstraction level để search: **phân bổ compute giữa các phần của detector**.

Sample Redistribution tác động vào đầu

- ❶ Mỗi ảnh train được crop vuông theo một scale rồi resize về cùng input 640×640 .
- ❷ Vì vậy cùng một khuôn mặt sau resize có thể lớn hơn hoặc nhỏ hơn tương đối, dù ảnh đầu vào là cùng một ảnh.
- ❸ Kích thước tương đối đó quyết định mặt rơi vào feature level nào và có bao nhiêu positive anchors.
- ❹ Do đó SR không chỉ là augmentation; nó đang điều khiển trực tiếp **training signal** cho tiny/small faces.

Luồng nhân quả

scale set $\rightarrow$ face size sau resize $\rightarrow$ positive samples $\rightarrow$ Hard AP

Giả định và giới hạn của SR

- SR cũng không tối ưu trực tiếp positive supervision; paper chỉ search trên một **tập scale rời rạc** rồi để pipeline crop/resize biến nó thành tín hiệu học.
- Nghĩa là paper tối ưu một biến gián tiếp: scale set, chứ không tối ưu trực tiếp số positives hay gradient mass theo level.
- Public release còn cho thấy narrative của paper và code public không hoàn toàn trùng nhau: code công khai giữ một scale list hand-crafted, trong khi paper nhấn mạnh searched scale set.
- Điểm này rất quan trọng khi đọc kết quả reproduction: chênh lệch Hard AP có thể đến từ chính sample policy, không chỉ từ backbone hay random seed.

Điểm mạnh và hạn chế của SCRFD

Điểm mạnh

- Đặt đúng bài toán tiny-face detection dưới ràng buộc hiệu quả.
- Nối được kiến trúc và dữ liệu train thành một pipeline logic.
- Đường cong accuracy-efficiency rất thực dụng cho face detection.

Hạn chế

- Search vẫn tốn tài nguyên.
- Search space còn bị ràng buộc bởi template detector ban đầu.
- Kết quả gắn rất mạnh với WIDER FACE và tiny-face benchmark.

Reproduction

WIDER FACE là benchmark trung tâm của báo cáo

Split	Số ảnh	Số face boxes	Ghi chú
Train	12,880	159,393	dùng để train
Val	3,226	39,697	dùng để evaluate
Test	16,097	–	không có GT public

- Paper thực chất bám rất chặt vào WIDER FACE để search, train và benchmark.
- EDA trong report cho thấy hơn 70% face boxes nằm dưới ngưỡng 32 px, nên tiny faces không phải edge case mà là vùng dữ liệu thống trị bài toán.

Cấu hình baseline chạy lại paper

- Nhóm bám **official public SCRFD codebase** và official detector family, không tự viết lại mô hình.
- Baseline được chọn là **SCRFD-2.5GF**: đủ lớn để có ý nghĩa thống kê, nhưng vẫn nằm trong phạm vi tài nguyên có thể tái hiện ổn định.
- Đây là quyết định có chủ ý: full NAS của paper quá nặng vì phải sinh nhiều candidate, train rút gọn, đánh giá, chọn cấu hình rồi mới train dài cho model cuối.
- Scale policy **paper-SR12** được nhóm **phục dựng lại** trên pipeline công khai; nó không phải một baseline upstream có sẵn đúng nguyên xi từ authors.
- Vì vậy reproduction của nhóm là **public reproducible baseline trên official stack**, không phải rerun toàn bộ NAS end-to-end của paper.

Protocol train và evaluate của baseline

Thành phần	Thiết lập
Input	640×640
Optimizer	SGD, momentum 0.9, weight decay 5×10^{-4}
Schedule	80 epochs, cosine LR, warmup
Augmentation	crop vuông, resize, flip, photometric distortion, normalize
Evaluation	WIDER FACE Easy/Medium/Hard trên validation split , threshold 0.02, NMS 0.45

- Điểm cần nhớ là baseline dùng cùng pipeline train/eval với official repo, nhưng đường chạy '80e cosine + paper-SR12' là cấu hình thực nghiệm nhóm dựng lại trên official stack.
- Vì vậy khi so với official model zoo, phải tách rõ hai lớp nguyên nhân: khác recipe train dài-ngắn và khác sample policy.

Kết quả tái hiện baseline và vai trò của paper-SR12

Thiết lập	Easy	Medium	Hard	mAP
Official model zoo	93.78	92.16	77.87	87.94
Baseline default scale set	91.40	89.69	72.49	84.53
Baseline paper-SR12	91.61	89.99	73.71	85.10

- Trong cùng khung **80 epoch cosine**, chỉ riêng việc đổi sang **paper-SR12** đã kéo Hard AP tăng +1.22.
- Điều này xác nhận một insight lớn của report: phần khó nhất của SCRFD không chỉ là chạy model, mà là tái tạo đúng sample policy cho tiny faces.

Đọc đúng gap với official model zoo

- Gap với official không nên quy hết cho một lý do duy nhất.
- **Official 2.5GF** là mốc model zoo đã được huấn luyện rất dài trong public recipe gốc, khoảng **640 epoch với step schedule**.
- **Baseline của nhóm là 80 epoch với cosine scheduler** trên official public codebase, rồi gắn thêm paper-SR12 vào pipeline công khai.
- Vì vậy gap Hard -4.16 phải được đọc như tổng hợp của:

Bốn nguồn chênh lệch

training horizon, scale policy, randomness, và mức độ tinh chỉnh sâu hơn của pipeline official

SPOS-NAS Cho CR

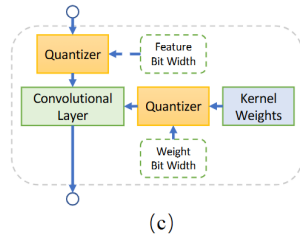
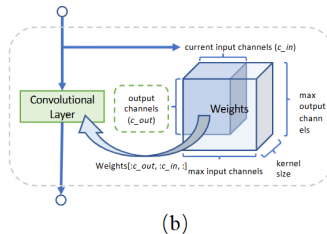
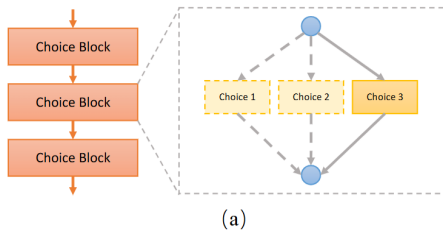
Vì sao dùng SPOS-NAS để mở rộng CR

- Official CR search rất nặng, nên nhóm dùng **SPOS-NAS proxy** để khảo sát sâu hơn cấu trúc phân bổ compute trong cùng miền 2.5 GFLOPs.
- Mục tiêu không phải thay official AP benchmark, mà là kiểm tra xem khi search dưới áp lực tiny-face detection, winner có tự học một quy luật phân bổ compute nhất quán hay không.
- Đây là **phân tích bổ sung cho CR**, không phải nhánh evidence chính mạnh hơn baseline + ASR.
- Đây là phần mở rộng trực tiếp cho **Computation Redistribution**, nên được trình bày trước.

Flow triển khai SPOS-NAS proxy

- 1 Xây dựng một supernet bao phủ nhiều phương án depth/width trên bốn stage $C_2 \rightarrow C_5$.
- 2 Huấn luyện supernet theo kiểu single-path để mỗi mini-batch chỉ kích hoạt một đường con.
- 3 Sau khi supernet đủ ổn định, lấy mẫu **1,000** candidate trong cùng vùng 2.5 GFLOPs.
- 4 Với mỗi candidate, chạy BatchNorm recalibration rồi đo proxy score trên validation.
- 5 Phân tích cấu hình Top-10 / thua để rút ra quy luật.

Minh họa luồng làm việc của SPOS

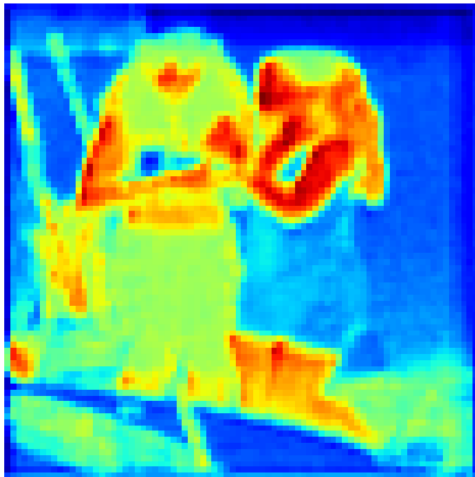


Inference heatmap của kiến trúc tốt nhất

Ảnh gốc



Bản đồ nhiệt dự đoán (Heatmap)

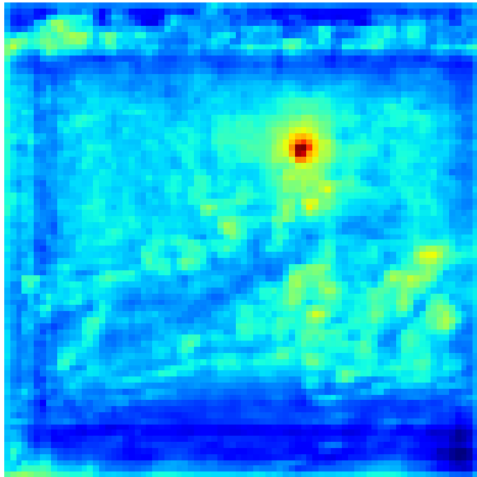


Ví dụ heatmap generated theo edge-focus

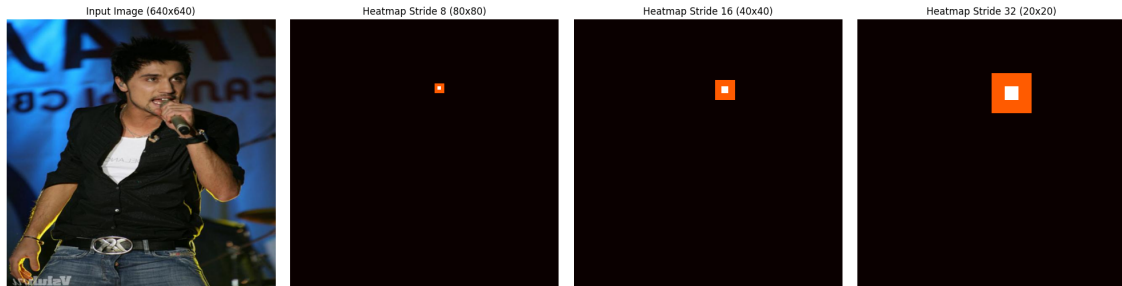
Ảnh gốc



Bản đồ nhiệt dự đoán (Heatmap)



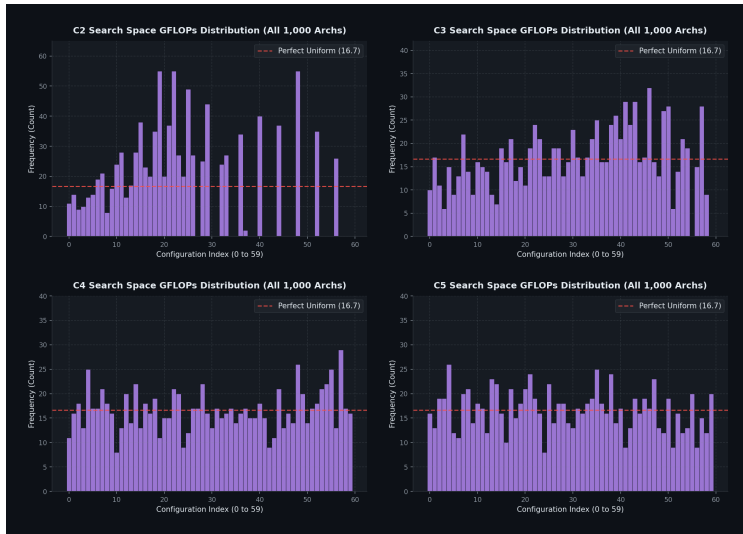
Ví dụ heatmap ngược lại theo center-focus



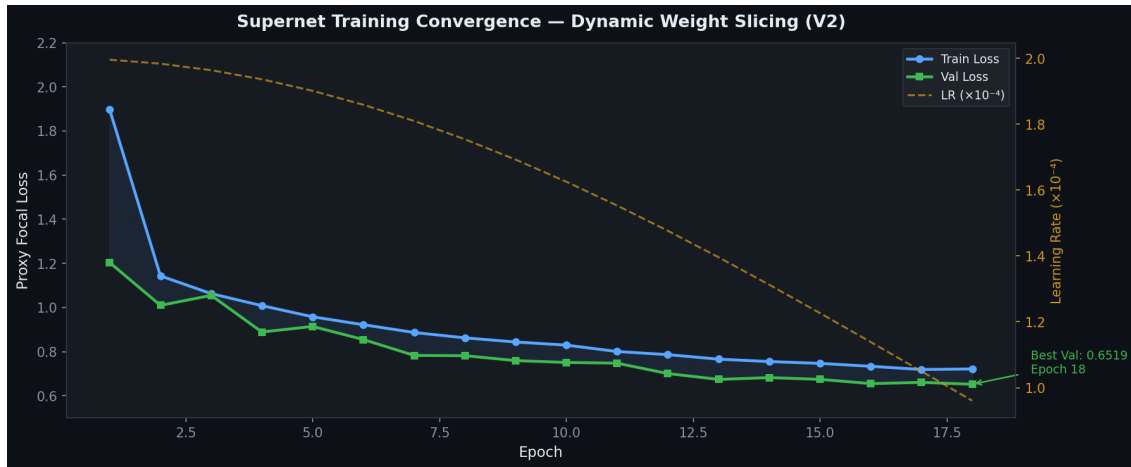
Đánh giá Supernet và Proxy NAS

- Cả supernet center-focus và edge-focus đều hội tụ ổn định sau khi giảm LR.
- Quá trình lấy mẫu sinh ra không gian biến thể bao phủ rất đều đặn, rải rác công bằng cơ hội thay đổi cấu trúc cho mọi tầng.
- Bảng xếp hạng 1,000 candidates cho thấy sự phân hóa rõ rệt dù cùng GFLOPs, tạo cơ sở mạnh để chọn lọc cấu hình tối ưu.

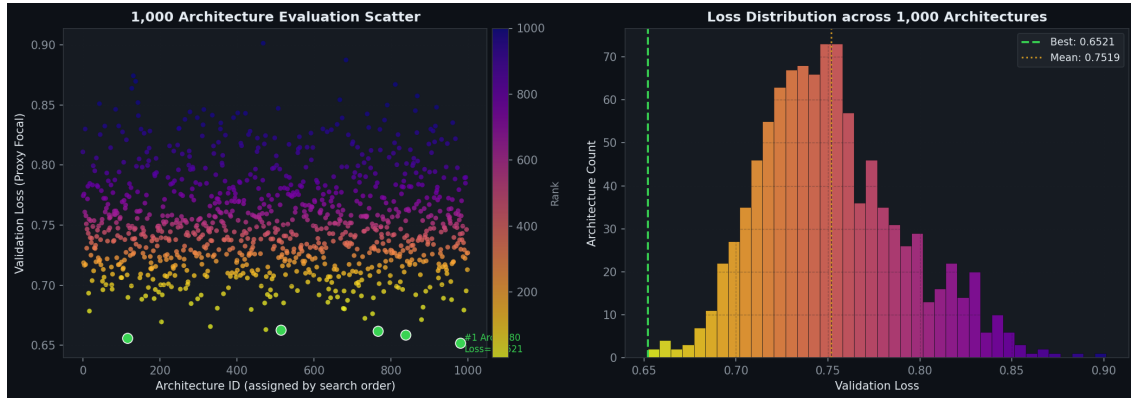
Tính đồng đều của không gian Search (Uniformity)



Đường cong hội tụ của Supernet



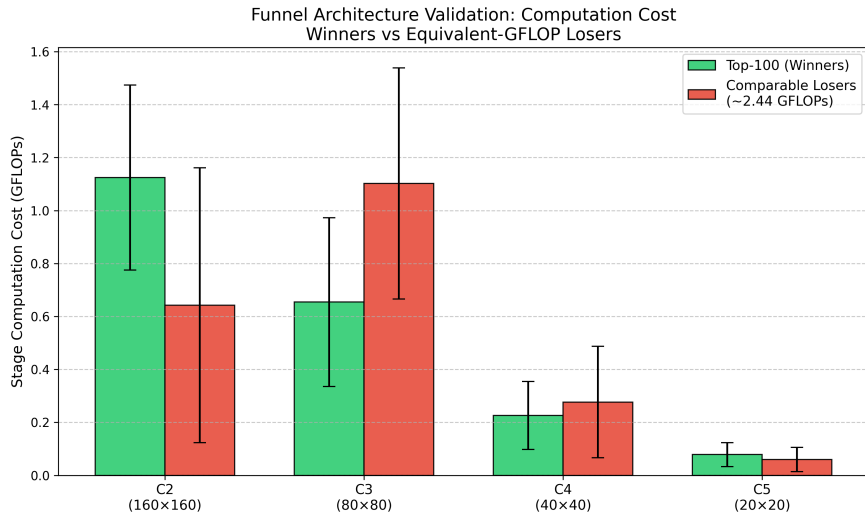
Leaderboard 1,000 Candidate



Kết quả chính của SPOS-NAS

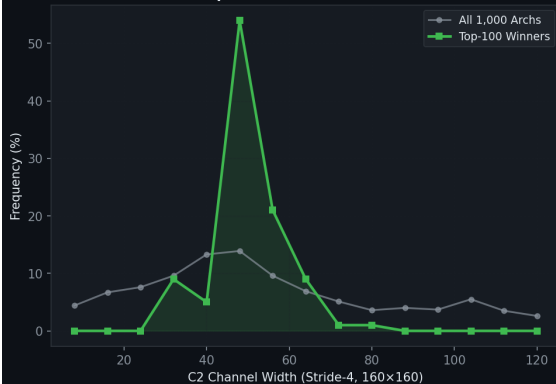
- Nhóm tốt có xu hướng **đầu tư mạnh vào** C_2 và tránh làm C_3 phình quá mức.
- C_2 giữ chi tiết vị trí không gian cho tiny faces, còn C_5 bổ sung ngữ nghĩa đủ mạnh cho head.
- C_3 là vùng dễ thành bẫy chi phí: feature map vẫn còn lớn nên chi phí đắt, nhưng gain tăng không tương xứng.
- Khi đổi proxy từ center-focus sang edge-focus, phân bố học dịch chuyển (giảm C_2 tăng C_3) → phản ứng linh hoạt với hình học của bài toán (vì ở viền cần thêm ngữ cảnh không gian nên cần stride lớn hơn).

Phân tích cơ cấu phân bố: Tránh bẫy FLOPs ở C3

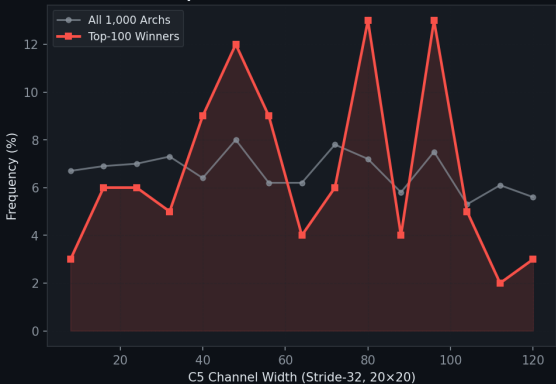


Top winners dành compute cho C2 và C5

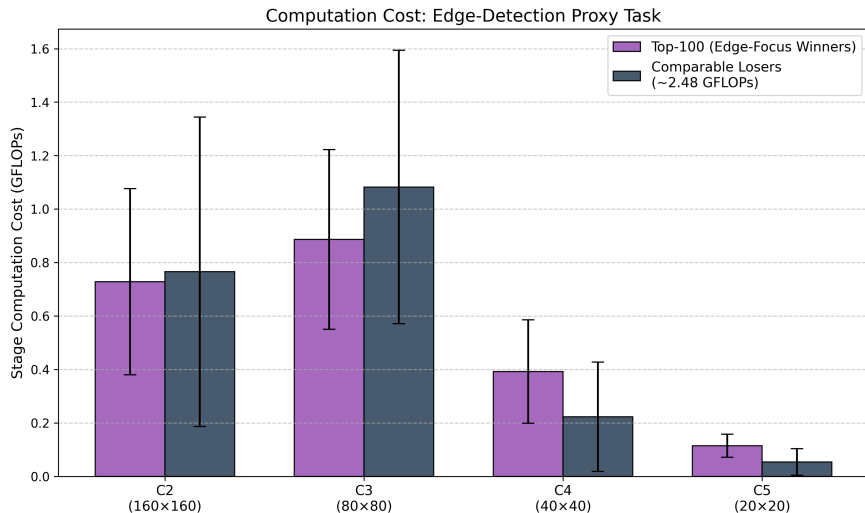
SR Effect: C2 Width Preference
Top-100 vs All Architectures



CR Effect: C5 Width Preference
Top-100 Winners Prefer Thinner C5



Phân tích cơ cấu phân bố: Tập trung vào biên (Edge-focus)



Điểm mà phần mở rộng này xác nhận

Computation Redistribution của SCRFD là một nguyên tắc phân bổ compute có cấu trúc, không phải mẹo thử-sai ngẫu nhiên.

- Trong miền 2.5 GFLOPs, winner thắng vì phân bổ compute đúng hơn, không phải vì đơn giản lớn hơn.
- Đó là lý do report kết luận SCRFD mạnh vì nó đặt năng lực biểu diễn đúng chỗ ngay từ cấp kiến trúc.

ASR+JSAR Cho SR Và Train

- Sau baseline paper-SR12, gap lớn nhất vẫn nằm ở split Hard.
- Điều đó cho thấy sample policy tĩnh vẫn chưa đủ, và tiny faces vẫn còn bị thiếu positive supervision ở bước assignment.
- Vì vậy nhóm sửa đúng hai nút thắt của giai đoạn train:

Hai cải tiến

ASR: điều chỉnh xác suất lấy các scale theo tín hiệu khó dễ đang quan sát được.

JSAR: tăng positive assignment cho tiny faces khi pipeline chuẩn còn bỏ sót.

ASR và JSAR hoạt động như thế nào

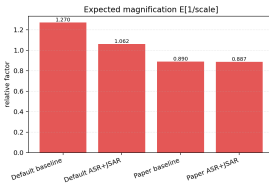
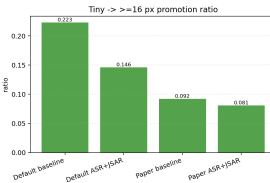
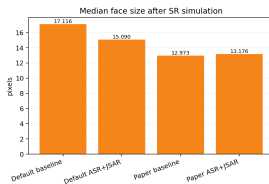
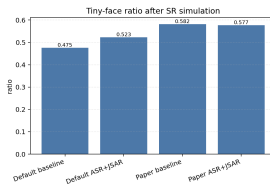
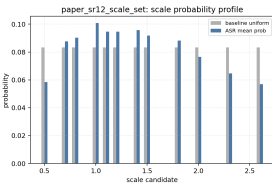
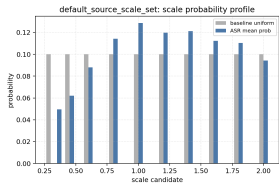
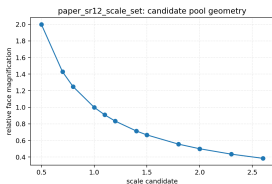
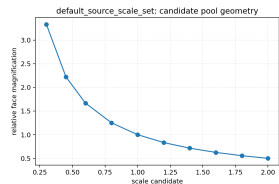
ASR

- Bắt đầu từ scale pool đủ rộng.
- Gom thống kê face-size distribution và thiếu hụt recall theo từng giai đoạn train.
- Tăng xác suất cho các scale tạo ra nhóm tiny/small faces còn thiếu.
- Làm mượt policy bằng EMA để tránh dao động quá mạnh.

JSAR

- Tiny faces thường nhận quá ít positives với assignment gốc.
- JSAR nói lỏng bước chọn ứng viên quanh tâm.
- Nếu vẫn thiếu positives, fallback sẽ giữ lại một nhóm anchor gần nhất còn hợp lý.
- Nhờ đó tiny faces không còn bị rơi khỏi supervision.

Scale pool và áp lực phân phối của ASR



Protocol thực nghiệm cho ASR+JSAR

- Cả baseline và cải tiến đều dùng **SCRFD-2.5GF**, input 640×640 , 80 epoch, cosine scheduler.
- Nhóm chạy hai cặp thí nghiệm để tách ảnh hưởng của sample pool:

Hai trường hợp

Default scale set: baseline gốc của source code.

Paper-SR12 scale set: scale policy gần hơn với narrative của paper.

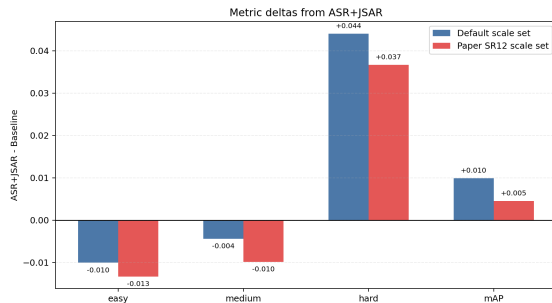
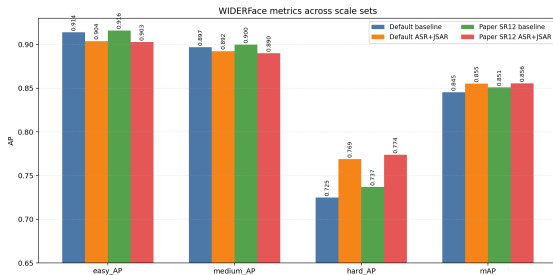
- Cách so sánh này giúp đọc đúng delta của ASR+JSAR: gain đến từ adaptive redistribution và assignment, không chỉ từ việc đổi scale pool.
- Trên nhánh 'default', baseline và ASR không dùng hoàn toàn cùng điểm đầu bên trái của scale pool; trên nhánh **paper-SR12**, cả hai mới thật sự dùng cùng một 12-scale set.

Kết quả ASR+JSAR trên WIDER FACE

Thiết lập	Easy	Medium	Hard	mAP
Default baseline	91.40	89.69	72.49	84.53
Default ASR+JSAR	90.39	89.25	76.90	85.51
Paper-SR12 baseline	91.61	89.99	73.71	85.10
Paper-SR12 ASR+JSAR	90.28	89.01	77.38	85.56

- Gain lớn nhất tập trung ở **Hard**: +4.40 với default và +3.67 với paper-SR12.
- Trên paper-SR12, kết quả chỉ còn cách official model zoo khoảng 0.49 điểm ở split khó nhất.

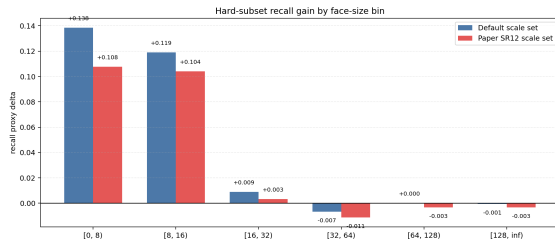
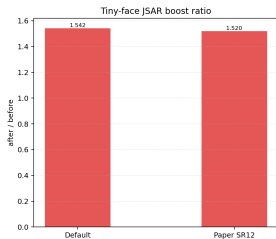
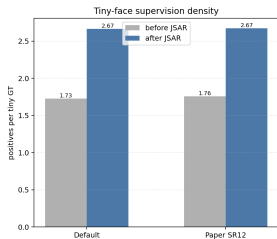
Metric và delta của ASR+JSAR



ASR+JSAR đang sửa đúng cái gì

- Về **dữ liệu train**, ASR làm thay đổi face-size distribution theo hướng detector còn thiếu.
- Về **supervision**, JSAR làm tiny positives/GT tăng từ khoảng 1.76 lên 2.67, tức gần $1.52\times$.
- Về **kết quả**, gain tập trung ở Hard subset, nghĩa là phần cải tiến đúng là đang chạm vào vùng tiny/hard faces thay vì cải thiện đồng đều mọi split.
- Trade-off vẫn tồn tại: Easy/Medium giảm nhẹ, nhưng đó là đánh đổi có chủ đích để đẩy training signal về đúng nhóm khó nhất.

JSAR và gain theo nhóm khuôn mặt khó



Kết luận

- SCRFD mạnh vì tối ưu đồng thời **nơi mô hình đặt năng lực biểu diễn** và **nơi dữ liệu tạo tín hiệu học**.
- **Baseline reproduction** cho thấy gap lớn nhất luôn nằm ở Hard; đó mới là vùng khó thật của SCRFD.
- **ASR+JSAR** là bằng chứng mạnh nhất của report: giữ nguyên kiến trúc suy luận vẫn có thể kéo Hard AP từ 73.71 lên 77.38 nếu scale distribution và positive supervision cho tiny faces được sửa đúng chỗ.
- **SPOS-NAS** củng cố phía CR: trong cùng miền 2.5 GFLOPs, winner thắng vì phân bổ compute đúng hơn, đặc biệt ở C_2 , C_3 và C_5 .
- Insight cuối cùng của report là: với efficient face detection, tối ưu kiến trúc thôi là chưa đủ; phải tối ưu cùng lúc **architecture, data distribution và supervision**.

Cảm ơn thầy và các bạn đã lắng nghe

Trả lời câu hỏi